Assignment 1: Java Foundations & Flow Control (Practical)

Practical Coding Exam: 2 Programming Challenges (Easy & Medium)

Total Questions: 2 Practical	**Difficulty: Easy & Medium**	**Format: Hands-on Source Code**	**Compiler: JDK 17+**

Submission & Execution Rules:
• All solutions must be compilable, working Java source files (.java) executing cleanly with javac and java.
• Validate user input, avoid unhandled runtime exceptions, and provide formatted console output as demonstrated.
• Each question must contain clear comments explaining the chosen algorithm and data structure.

[EASY LEVEL] — Scientific Temperature & Metric Telemetry Converter PRACTICAL LAB

Problem Statement: Write a standalone program MetricConverter.java that prompts the user for temperature in Celsius and distance in kilometers, then converts them using exact arithmetic and explicit type casting while preventing integer division truncation.

Technical Requirements & Implementation Checklist:

• Convert Celsius to Fahrenheit using formula: F = (C * 9.0 / 5.0) + 32.0 (ensure no integer division loss).

• Convert Celsius to Kelvin: K = C + 273.15. Check that temperature does not fall below absolute zero (-273.15°C); print an error if invalid.

• Convert Kilometers to Miles (1 km = 0.621371 miles) and Nautical Miles (1 km = 0.539957 nmi).

• Format all output floating-point values to exactly 2 decimal places using System.out.printf().

Starter Code Template:

```
import java.util.Scanner;
public class MetricConverter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter temperature in Celsius: ");
        double celsius = sc.nextDouble();
        // TODO: Validate absolute zero and compute conversions
    }
}
```

Sample Console Interaction:

```
Enter temperature in Celsius: 37.0
Enter distance in Kilometers: 100.0
--- CONVERSION TELEMETRY ---
Temperature: 37.00 °C = 98.60 °F = 310.15 K
Distance:    100.00 km = 62.14 miles = 54.00 nautical miles
```

Evaluation Criteria: *Validation & Boundary Check | Correct Math & No Truncation | Formatted Console Output*

[MEDIUM LEVEL] — Smart Cash Register & Optimal Currency Dispenser PRACTICAL LAB

Problem Statement: Implement CashRegister.java to model a retail point-of-sale checkout. The program accepts the total purchase amount and the cash tendered by the customer, validates sufficient payment, and dispenses change using the absolute minimum count of bills and coins.

Technical Requirements & Implementation Checklist:

• Prompt for total bill amount and cash tendered. If cash tendered < bill, print the deficit amount and request additional cash in a loop until paid.

• Denominations supported: $100, $50, $20, $10, $5, $1 bills, and 25¢ (quarters), 10¢ (dimes), 5¢ (nickels), 1¢ (pennies).

• Use integer arithmetic (convert total dollars to cents: totalCents = Math.round(amount * 100)) to avoid IEEE 754 floating-point rounding errors.

• Display an itemized tabular breakdown listing only denominations with count > 0, followed by the total physical units dispensed.

Starter Code Template:

```
import java.util.Scanner;
public class CashRegister {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Total Bill ($): ");
        double bill = sc.nextDouble();
        // TODO: Loop for sufficient cash and compute change in cents
    }
}
```

Sample Console Interaction:

```
Enter Total Bill ($): 63.42
Enter Cash Tendered ($): 100.00
Change Due: $36.58
  - $20 Bills: 1
  - $10 Bills: 1
  - $5 Bills:  1
  - $1 Bills:  1
  - Quarters (25c): 2
  - Nickels  (5c):  1
  - Pennies  (1c):  3
Total items dispensed: 10
```

Evaluation Criteria: *Cash loop & deficit validation | Integer cents conversion preventing precision bugs | Clean table formatting*